Facultad de Ingeniería

TRABAJO PRÁCTICO

Inteligencia Artificial

Módulo [N]

Actividad: Redes Neuronales Convolucionales (CNN)

Legajo	Apellido y Nombre	e-mail
0129980	Zaragoza Yago	yagozaragoza04@gmail.com

Carrera: Ingeniería Informática

Modalidad: Asincrónico

Docente: Ignacio Casais

Fecha de entrega: [DD/MM/AAAA]

Consigna

Construir un modelo e implementar utilizando CNN. Para ello debés:

a. Buscar y seleccionar un conjunto de datos para trabajar (Kaggle / Google Dataset Search).

b. Programar (construir) la red neuronal.

c. Realizar el proceso de aprendizaje.

d. Realizar la validación del modelo utilizando ejemplos y cotejando el resultado.

1. Introducción y objetivo

Una red neuronal convolucional (CNN) es una red neuronal que usa la convolución, un filtro o kernel que recorre la imagen, en lugar de la multiplicación matricial general en al menos una de sus capas, y la combina con capas ReLU, de agrupación (pooling), flatten y totalmente conectadas (material de cátedra: PPT Redes Neuronales Convolucionales). El objetivo de este trabajo es construir, entrenar y validar una CNN que clasifique imágenes de prendas del conjunto Fashion-MNIST en sus 10 clases.

El informe sigue el orden de la consigna. La sección 2 (consigna a) justifica la selección del conjunto de datos y describe su preprocesamiento; la sección 3 (consigna b) presenta la arquitectura de la red; la sección 4 (consigna c) explica el proceso de aprendizaje, y la sección 5 (consigna d) valida el modelo sobre el conjunto de prueba oficial con métricas por clase, ejemplos cotejados, el análisis de los errores y la comparación con una red sin convolución y con resultados publicados. Los anexos documentan el entorno y la ejecución (Anexo A), los ejemplos cotejados (Anexo B), el detalle por época (Anexo C) y la verificación de los datos (Anexo D).

2. Consigna a: selección del conjunto de datos

Búsqueda y criterios

La búsqueda y la descarga se realizaron en Kaggle, con seis criterios: que los datos fueran imágenes, insumo natural de una red neuronal convolucional; que el etiquetado fuera

confiable y hecho por expertos; que las clases estuvieran balanceadas; que el tamaño permitiera entrenar sin GPU; que la licencia fuera abierta, y que existiera un benchmark publicado contra el cual cotejar los resultados.

Se eligió Fashion-MNIST, de Zalando Research, publicado con licencia MIT en Kaggle y en GitHub (repositorio oficial de Fashion-MNIST). Sus etiquetas provienen del código de silueta de cada producto, que fue etiquetado manualmente por expertos en moda de Zalando y revisado por otro equipo de la empresa (Xiao, Rasul y Vollgraf, 2017). Se descartó el MNIST original de dígitos manuscritos porque es el ejemplo que desarrolla el video de la cátedra (material de cátedra: video M9); Fashion-MNIST fue creado como reemplazo directo y más exigente de MNIST y conserva su formato, por lo que la arquitectura vista en clase se aplica sin cambios a un problema menos trivial.

Descripción del conjunto

El conjunto contiene 70.000 imágenes de prendas y accesorios en escala de grises de 28 × 28 píxeles: 60.000 de entrenamiento y 10.000 de prueba, repartidas en 10 clases (Tabla 1). Las clases están balanceadas: 6.000 imágenes por clase en entrenamiento y 1.000 en prueba (Figura 2). La Figura 1 muestra ejemplos de cada clase.

Tabla 1. Clases de Fashion-MNIST: nombre usado en este trabajo y nombre original (Xiao, Rasul y Vollgraf, 2017, Tabla 2).

N.º	Nombre en español	Nombre original (inglés)
0	Remera/top	T-Shirt/Top
1	Pantalón	Trouser
2	Buzo	Pullover
3	Vestido	Dress
4	Abrigo	Coat
5	Sandalia	Sandals
6	Camisa	Shirt
7	Zapatilla	Sneaker
8	Bolso	Bag
9	Bota	Ankle boots



Figura 1. Seis ejemplos por clase del conjunto de entrenamiento, elegidos al azar con semilla fija. Visualización invertida (prenda oscura sobre fondo claro); el modelo recibe la imagen original, con la prenda clara sobre fondo negro.

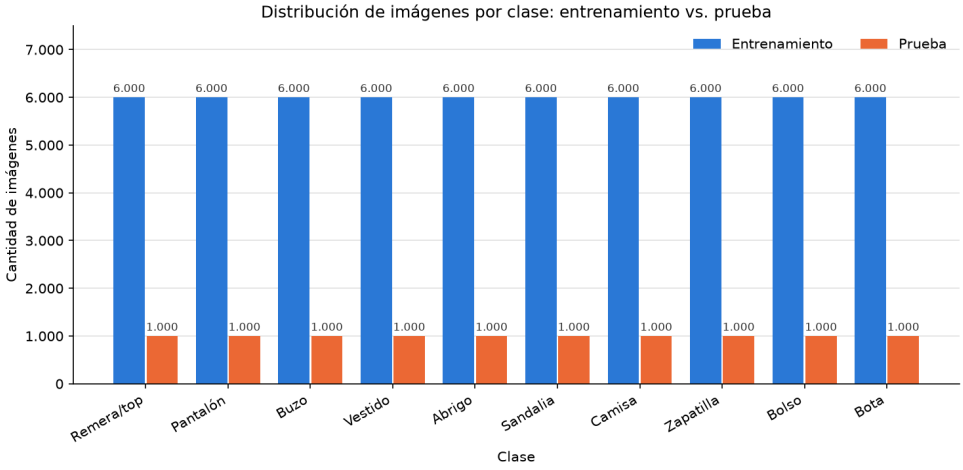


Figura 2. Cantidad de imágenes por clase en los conjuntos de entrenamiento y prueba.

ATENCIÓN - los archivos CSV de la descarga no son la partición oficial

La descarga de Kaggle trae el conjunto en formato CSV y en formato idx. Al cotejarlos imagen por imagen se comprobó que los CSV no reproducen la partición oficial: su conjunto de entrenamiento incluye 7.885 imágenes del conjunto de prueba oficial y 4.813 ausentes de él, y comparte 10 imágenes con su propio conjunto de prueba.

Mediante sumas MD5 se verificó que los archivos idx son idénticos a los del repositorio oficial, por lo que se trabajó con ellos: solo con la partición oficial los resultados son comparables con el benchmark. El detalle está en el Anexo D.

Preprocesamiento

El preprocesamiento tuvo tres pasos: dividir los píxeles por 255 para llevarlos al intervalo [0, 1]; dar a cada imagen la forma 28 × 28 × 1, con un único canal de intensidad, y codificar las etiquetas en formato one-hot con to_categorical, como en el ejemplo de la cátedra, de modo que cada clase queda representada por un vector de 10 posiciones.

3. Consigna b: construcción de la red

La red es un modelo secuencial (Sequential) de tensorflow.keras que sigue la red convolucional sobre MNIST del video de la cátedra, con un bloque convolucional más (Figura 3 y Tabla 2).

Rol de cada capa

La entrada es una imagen de 28 × 28 píxeles con un canal. En cada capa de convolución, un filtro o kernel de 3 × 3 recorre la imagen: en cada posición multiplica elemento a elemento sus pesos por la región que cubre, suma los productos y agrega un sesgo. Ese valor es una neurona del mapa de características; como el mismo filtro recorre toda la imagen, sus pesos se comparten. Las convoluciones usan la función de activación ReLU, f(x) = max{0, x}, que anula los valores negativos (material de cátedra: PPT Redes Neuronales Convolucionales).

Cada bloque termina con una agrupación máxima (max pooling) de 2 × 2, que conserva el máximo de cada grupo de cuatro neuronas y reduce a la mitad el alto y el ancho; en pool2, la entrada de 11 × 11 queda en 5 × 5 porque se descartan la última fila y la última columna, impares. La capa flatten convierte esos mapas (5 × 5 × 64) en un vector 1D de 1.600 valores, que alimenta una capa totalmente conectada de 128 neuronas, donde se realiza el razonamiento de alto nivel. El dropout descarta al azar una fracción de las activaciones para

reducir el sobreajuste; actúa solo durante el entrenamiento, y en la predicción se usan todas las neuronas. La salida tiene 10 neuronas con softmax, que entrega una probabilidad por clase.

Decisiones de diseño

El segundo bloque opera sobre mapas ya reducidos y combina características de mayor nivel. Los filtros crecen de 32 a 64 mientras el pooling reduce la resolución, lo que suma capacidad sin multiplicar el costo. La primera convolución de cada bloque usa padding 'same' para conservar el tamaño y la información del borde; la segunda recorta un píxel por lado. El dropout (25 % tras cada bloque y 50 % tras la capa densa) replica el ejemplo de la cátedra. En Keras 3 la forma de entrada se declara con keras.Input, que reemplaza al argumento input_shape que usa el video.

Función de pérdida y optimizador

El ejemplo XOR del video usa error cuadrático medio con una única neurona sigmoide, adecuada para una salida binaria. Aquí hay 10 clases excluyentes: softmax produce probabilidades que suman 1, y la entropía cruzada categórica penaliza la baja probabilidad asignada a la clase correcta, según la etiqueta one-hot. El optimizador es Adam.

Parámetros

La red tiene 271.210 parámetros, todos entrenables. La capa densa posterior al flatten concentra 204.928 (75,56 %); las cuatro convoluciones suman 64.992 (23,96 %), gracias a los pesos compartidos. El cálculo se verificó a mano con (kh·kw·canales + 1)·filtros = (3·3·1 + 1)·32 = 320 para conv1_1, y con (entradas + 1)·neuronas = (1.600 + 1)·128 = 204.928 para la capa densa. Una prueba sin entrenar sobre ocho imágenes devolvió una salida de 8 × 10 cuyas filas suman 1.

Tabla 2. Arquitectura de la red: forma de salida y parámetros de cada capa.

Capa	Tipo	Forma de salida	Parámetros
entrada	Input	28 × 28 × 1	0
conv1_1	Conv2D	28 × 28 × 32	320
conv1_2	Conv2D	26 × 26 × 32	9.248
pool1	MaxPooling2D	13 × 13 × 32	0
drop1	Dropout	13 × 13 × 32	0
conv2_1	Conv2D	13 × 13 × 64	18.496
conv2_2	Conv2D	11 × 11 × 64	36.928

Capa	Tipo	Forma de salida	Parámetros
pool2	MaxPooling2D	5 × 5 × 64	0
drop2	Dropout	5 × 5 × 64	0
flatten	Flatten	1.600	0
densa	Dense	128	204.928
drop3	Dropout	128	0
salida	Dense	10	1.290
Total			**271.210**

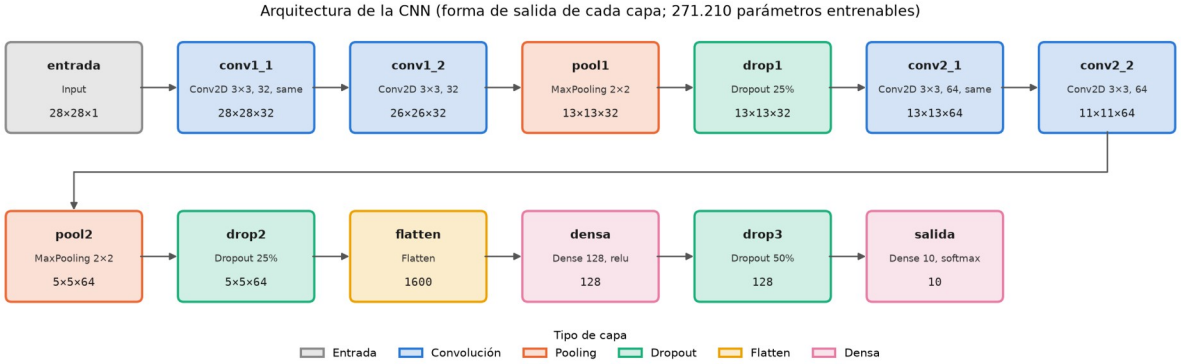


Figura 3. Arquitectura de la red: una caja por capa con su nombre, tipo y forma de salida; el color indica el tipo de capa.

La función que construye y compila la red (src/modelo.py) es la siguiente:

```
def construir_modelo():
    """Construye y compila la CNN.

    Fija la semilla antes, para que los pesos iniciales sean reproducibles.
    """
    datos.fijar_semilla()
    modelo = keras.Sequential(
        [
            # En Keras 3 la forma de entrada se declara con keras.Input; reemplaza al
            # argumento input_shape=(28, 28, 1) que el video pasa a la primera Conv2D.
            keras.Input(shape=FORMA_ENTRADA, name="entrada"),
            # Bloque convolucional 1: 32 filtros de 3x3
            layers.Conv2D(32, (3, 3), activation="relu", padding="same", name="conv1_1"),
            layers.Conv2D(32, (3, 3), activation="relu", name="conv1_2"),
            layers.MaxPooling2D((2, 2), name="pool1"),
            layers.Dropout(0.25, name="drop1"),
            # Bloque convolucional 2: 64 filtros de 3x3
            layers.Conv2D(64, (3, 3), activation="relu", padding="same", name="conv2_1"),
            layers.Conv2D(64, (3, 3), activation="relu", name="conv2_2"),
            layers.MaxPooling2D((2, 2), name="pool2"),
            layers.Dropout(0.25, name="drop2"),
            # Clasificador: vector 1D, capa totalmente conectada y salida de 10 clases
            layers.Flatten(name="flatten"),
            layers.Dense(128, activation="relu", name="densa"),
            layers.Dropout(0.5, name="drop3"),
            layers.Dense(datos.NUM_CLASES, activation="softmax", name="salida"),
        ],
        name="cnn_fashion_mnist",
    )
    modelo.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return modelo
```

4. Consigna c: proceso de aprendizaje

Aprendizaje supervisado y retropropagación

El aprendizaje es supervisado: se muestran a la red patrones de entrada junto con su salida deseada, y una fórmula de minimización del error ajusta los pesos para acercar ambas (material de cátedra: apunte Aprendizaje en las Redes Neuronales). Los pesos de filtros y capas densas parten de valores aleatorios. Para cada lote, la red calcula su salida, mide el error con la entropía cruzada y, mediante retropropagación (backpropagation), obtiene las derivadas parciales de ese error respecto de cada peso. Con ese gradiente, el optimizador Adam (variante del descenso de gradiente, con tasa de aprendizaje 0,001) ajusta pesos y filtros para reducir el error.

Épocas, lotes e iteraciones

Los 54.000 ejemplos de entrenamiento se procesan en lotes (batch) de 128, mezclados en cada época; cada lote produce una iteración, es decir, un ajuste de pesos. Una época, que es una pasada completa por los datos, tiene así 422 iteraciones: 421 lotes completos y uno de 112.

Validación y early stopping

Del entrenamiento oficial se separaron, de forma estratificada, 6.000 imágenes (600 por clase) para validación: la red no aprende de ellas; se usan para medir, al final de cada época, si lo aprendido generaliza. El early stopping vigila la pérdida de validación y, si no mejora en cinco épocas seguidas, detiene el entrenamiento y restaura los pesos de la mejor época. En una primera corrida, con un tope de 20 épocas y paciencia 3, la pérdida de validación oscilaba sin tendencia clara: empeoró respecto de su mínimo en las épocas 16, 18 y 19, pero nunca tres veces seguidas, de modo que el early stopping no llegó a actuar y la mejor época coincidió con el tope. Por eso se amplió el máximo a 50 y la paciencia a 5: así el corte responde a un estancamiento sostenido y no a una oscilación pasajera. El conjunto de prueba oficial no intervino en esta etapa (Tabla 3).

Tabla 3. Hiperparámetros del proceso de aprendizaje.

Parámetro	Valor
Optimizador	Adam (tasa de aprendizaje 0,001, valor por defecto)

Parámetro	Valor
Función de pérdida	Entropía cruzada categórica
Tamaño de lote	128
Épocas máximas	50
Early stopping	pérdida de validación, paciencia 5, restaura los mejores pesos
Mezcla de los datos	en cada época
Partición	54.000 de entrenamiento y 6.000 de validación (estratificada)
Iteraciones por época	422 (último lote de 112)
Semilla	42
Épocas ejecutadas / mejor época	26 / 21

Los callbacks y la llamada a fit (src/entrenar.py) son los siguientes:

```
if args.prueba:
    callbacks, epocas = [tiempo], 1
else:
    np.save(RUTA_INDICES_VAL, idx_val)
    callbacks, epocas = [
        tiempo,
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=PACIENCIA,
                                      restore_best_weights=True, verbose=1),
        keras.callbacks.ModelCheckpoint(str(RUTA_MEJOR), monitor="val_loss",
                                        save_best_only=True, verbose=1),
        keras.callbacks.CSVLogger(str(RUTA_LOG)),
    ], EPOCAS_MAX

inicio = time.perf_counter()
h = modelo.fit(x_entr, y_entr, validation_data=(x_val, y_val), batch_size=BATCH,
               epochs=epocas, shuffle=True, callbacks=callbacks, verbose=2)
```

Evolución del entrenamiento

La exactitud (accuracy) de validación pasó de 86,67 % en la época 1 a 94,03 % en la 21, y su pérdida de 0,368 a 0,175 (Figura 4 y Figura 5). Hasta esa época, la exactitud de entrenamiento queda por debajo, porque se promedia durante la época con dropout activo. Después las curvas se separan: la pérdida de entrenamiento sigue bajando (de 0,172 a 0,157) y la de validación no vuelve a su mínimo. Es el comienzo del sobreajuste: el early stopping cortó en la época 26 y restauró los pesos de la 21. El entrenamiento demoró 6,97 minutos en CPU, 16,04 s por época en promedio (detalle por época en la Tabla 8, Anexo C).

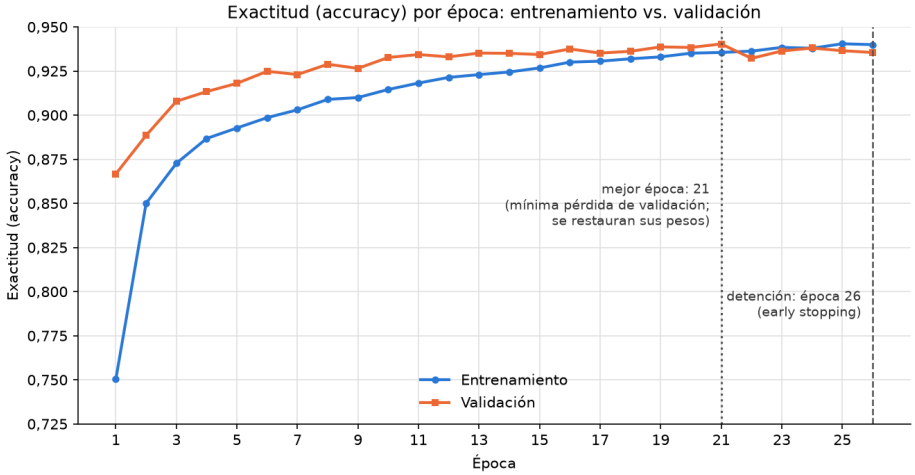


Figura 4. Exactitud (accuracy) de entrenamiento y validación por época. La línea punteada marca la mejor época (mínima pérdida de validación, cuyos pesos se restauran) y la discontinua, la época en que se detuvo el entrenamiento.

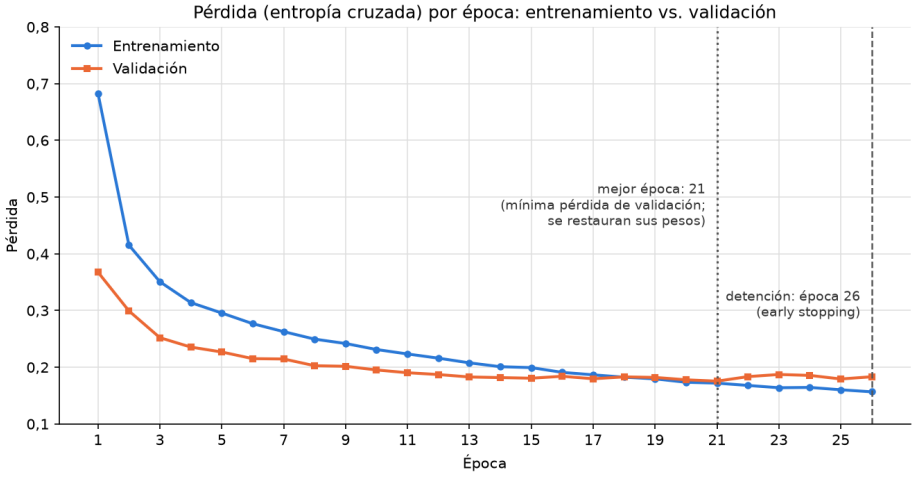


Figura 5. Pérdida (entropía cruzada) de entrenamiento y validación por época, con las mismas marcas que la Figura 4.

Mapas de activación

La Figura 6 muestra, para la imagen de prueba n.º 0 (bota), algunos mapas de la primera y de la última convolución del modelo entrenado. Los de conv1_1 (28 × 28) conservan la silueta y resaltan bordes y contornos; los de conv2_2 (11 × 11) concentran la activación en pocas zonas, como la franja horizontal de la suela: patrones más abstractos y de menor resolución, que la capa totalmente conectada combina para decidir la clase.

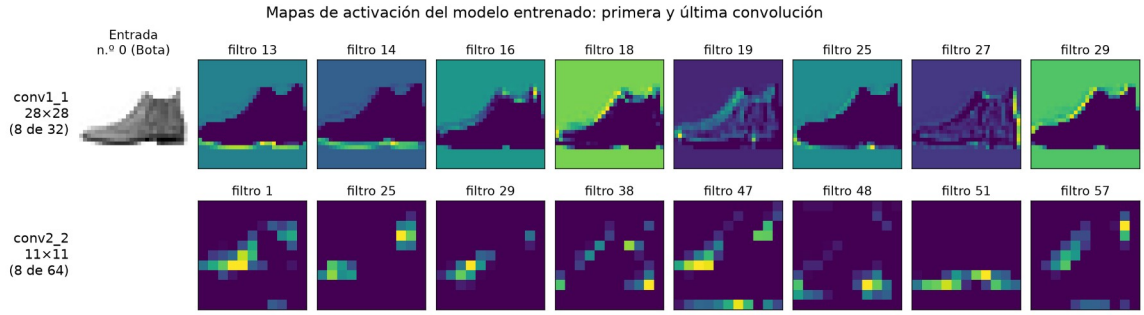


Figura 6. Imagen de prueba n.º 0 (bota) y ocho mapas de activación de conv1_1 (32 filtros) y de conv2_2 (64 filtros); se muestran los de mayor activación media. Visualización invertida de la entrada (prenda oscura sobre fondo claro); el modelo recibe la imagen original, con la prenda clara sobre fondo negro. Cada mapa usa su propia escala de color (más claro = mayor activación).

5. Consigna d: validación del modelo

Recién en esta etapa se usó el conjunto de prueba oficial: 10.000 imágenes que la red nunca vio. El modelo guardado obtuvo una exactitud de 93,06 % y una pérdida de 0,200, frente a 94,03 % y 0,175 en validación: 0,97 puntos porcentuales menos. La diferencia es chica, por lo que el modelo generaliza a datos nuevos; una leve caída es esperable, porque la mejor época se eligió mirando la validación.

Métricas por clase

La Tabla 4 resume la precisión (precision: qué parte de lo predicho como una clase es correcto), el recall (qué parte de cada clase se reconoce) y el f1-score, su media armónica. La mejor clase es Pantalón (f1 = 0,992) y la peor, Camisa (f1 = 0,792; precisión 0,775 y recall 0,810). Los promedios macro y ponderado coinciden en f1 = 0,931, porque las clases tienen el mismo soporte.

Tabla 4. Métricas por clase sobre el conjunto de prueba (classification_report de scikit-learn). Se resaltan la mejor y la peor clase según f1.

Clase	Precisión	Recall	F1-score	Soporte
Remera/top	0,889	0,880	0,884	1.000
Pantalón	**0,999**	**0,986**	**0,992**	**1.000**
Buzo	0,924	0,879	0,901	1.000
Vestido	0,919	0,936	0,928	1.000
Abrigo	0,889	0,901	0,895	1.000
Sandalia	0,991	0,984	0,987	1.000
Camisa	**0,775**	**0,810**	**0,792**	**1.000**
Zapatilla	0,957	0,980	0,968	1.000
Bolso	0,993	0,987	0,990	1.000

Clase	Precisión	Recall	F1-score	Soporte
Bota	0,978	0,963	0,970	1.000
Promedio macro	*0,931*	*0,931*	*0,931*	*10.000*
Promedio ponderado	*0,931*	*0,931*	*0,931*	*10.000*
Exactitud (accuracy)			**0,931**	**10.000**

Matriz de confusión

En la Figura 7, cada fila es la clase real y cada columna, la predicha. Los pares más confundidos son Remera/top → Camisa (83 casos), Camisa → Remera/top (82 casos) y Abrigo → Camisa (61). Los tres caen dentro del grupo de prendas de torso (remera/top, buzo, abrigo y camisa), que reúne 453 de los 694 errores (65,27 %): son siluetas parecidas que, a 28 × 28 píxeles, se distinguen por detalles como botones, cuello o largo de manga.

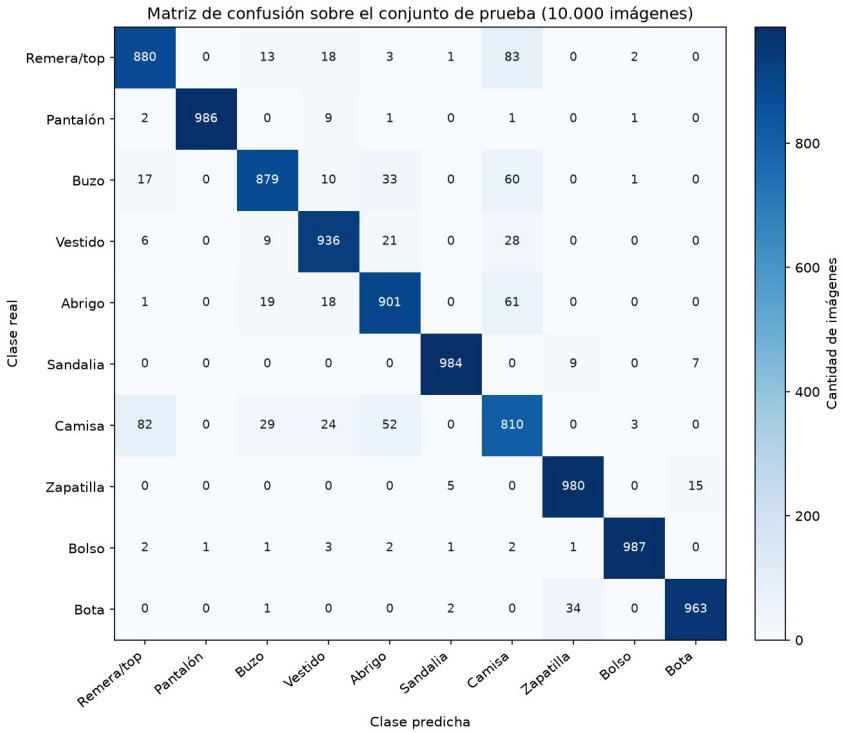


Figura 7. Matriz de confusión sobre las 10.000 imágenes de prueba: filas, clase real; columnas, clase predicha.

Ejemplos cotejados

Se eligieron 20 imágenes de prueba al azar antes de ver las predicciones (dos por clase, con semilla fija) y se cotejó la clase predicha con la real (Figura 8). El modelo acertó 18, en línea con los 18,61 aciertos esperables según la exactitud global. Los fallos fueron: dos casos de buzo predicho como camisa (p = 0,47 y 0,61), siempre con Remera/top como

segunda opción. El modelo duda, con probabilidades bajas, justamente entre prendas de torso. El detalle de cada ejemplo, con su top-2, está en el Anexo B (Tabla 7).

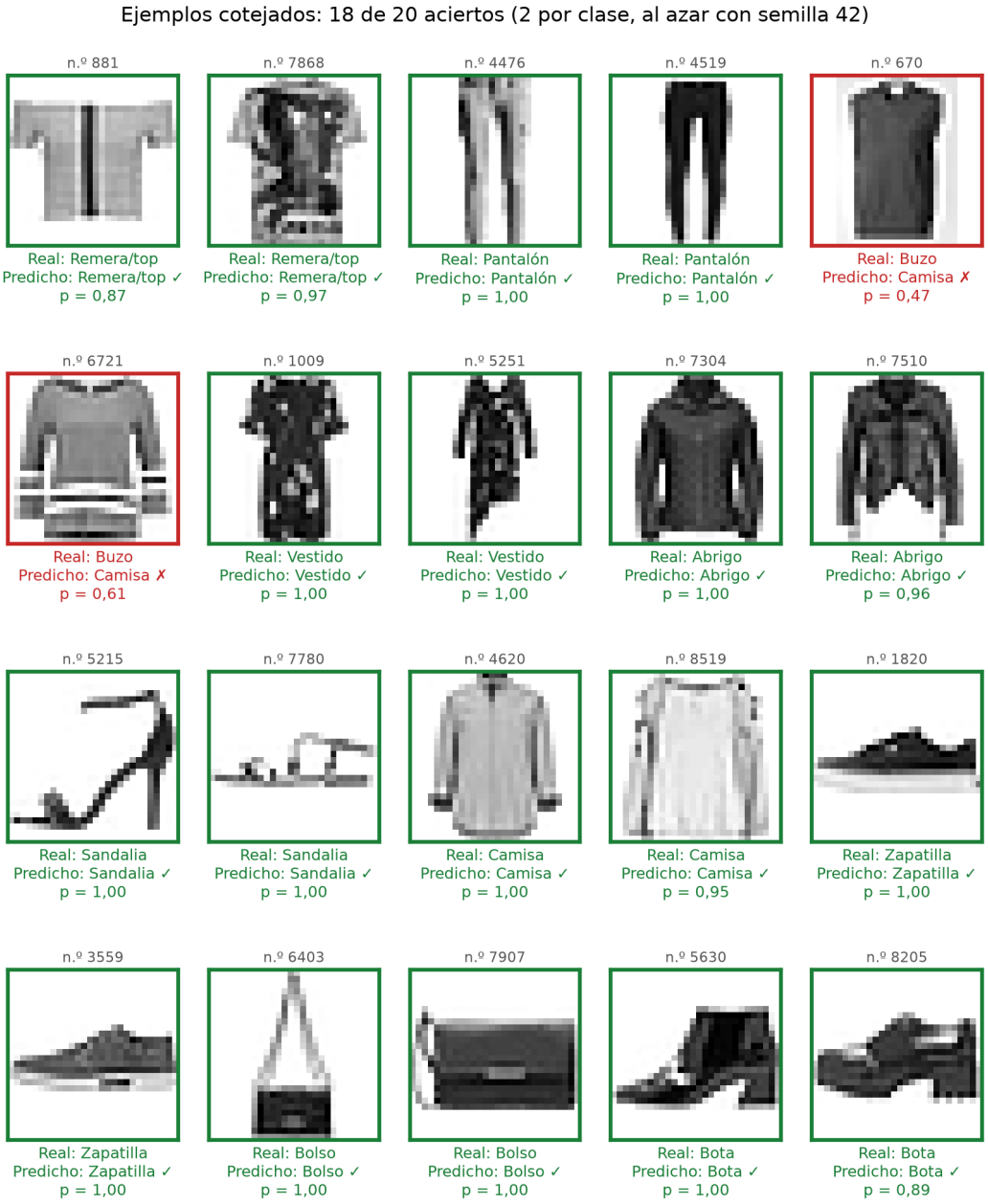


Figura 8. 20 ejemplos de prueba (dos por clase, al azar con semilla fija) con su clase real, la predicha y la probabilidad asignada. Verde y ✓: acierto; rojo y ✗: error. Visualización invertida (prenda oscura sobre fondo claro); el modelo recibe la imagen original, con la prenda clara sobre fondo negro.

Errores con alta confianza

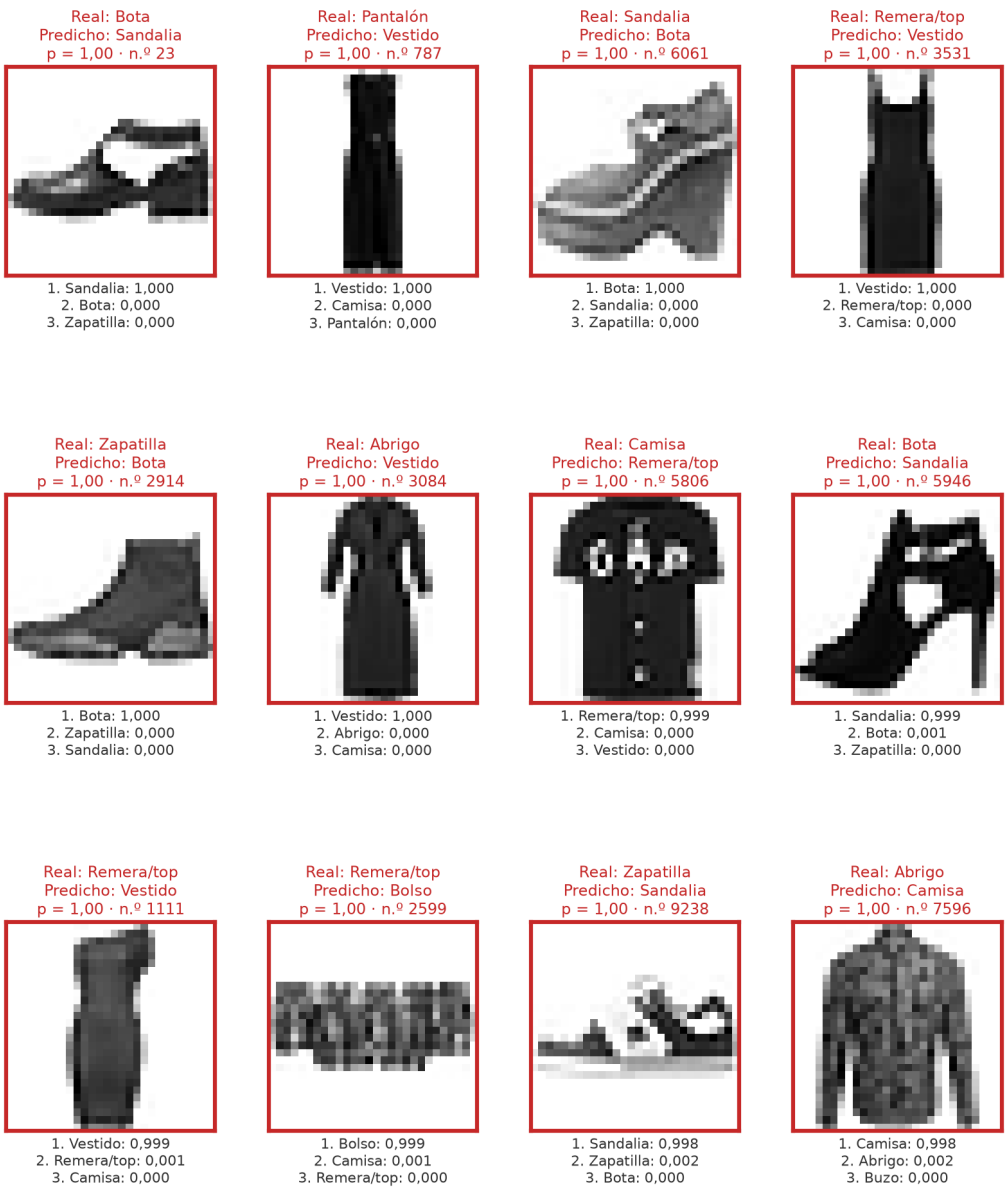


Figura 9. Los doce errores del conjunto de prueba con mayor probabilidad asignada a la clase equivocada, con su top-3. Visualización invertida (prenda oscura sobre fondo claro); el modelo recibe la imagen original, con la prenda clara sobre fondo negro.

De los 694 errores del conjunto de prueba, la Figura 9 muestra los doce con mayor confianza: todos asignan al menos 0,997 de probabilidad a la clase equivocada. A simple vista, la mayoría son entendibles: cinco son calzados con recortes o plataformas, ambiguos

entre bota, sandalia y zapatilla; cuatro son prendas largas y angostas (un abrigo, un pantalón, dos remeras/tops) leídas como vestido, y uno es una chaqueta estampada con botones tomada por camisa. Los otros dos parecen fallas del modelo o imágenes atípicas: camisa de manga corta con botones visibles, predicha como remera; remera que se ve como un rectángulo horizontal, predicha como bolso. Que el modelo se equivoque con probabilidades cercanas a 1 muestra que la salida softmax no equivale a certeza.

Aporte de la convolución y comparación con el benchmark

Para medir qué aporta la convolución se entrenó, en idénticas condiciones, una red sin convolución: un perceptrón multicapa que aplana la imagen y la pasa por una capa densa de 128 neuronas. Alcanzó 88,23 % con 101.770 parámetros, frente a 93,06 % de la CNN con 271.210: la convolución aporta 4,83 puntos porcentuales (Tabla 5). El resultado supera al mejor clasificador clásico del artículo original, un SVC con kernel RBF (89,7 %; Xiao, Rasul y Vollgraf, 2017, Tabla 3), y a dos CNN chicas de la tabla de contribuciones del repositorio oficial, «2 Conv+pooling» (91,6 %) y «3 Conv+pooling+2 FC+dropout» (92,6 %). Queda por debajo de «2 Conv+pooling+BN» (93,4 %), que agrega normalización por lotes. El MLP propio queda cerca del «MLP 256-128-100» informado allí (88,33 %). Las cifras de esa tabla las informan sus autores, en condiciones no homogéneas (repositorio oficial de Fashion-MNIST).

Tabla 5. Comparación de la CNN con la red sin convolución y con resultados publicados (exactitud sobre el conjunto de prueba oficial; s/d: sin dato en la fuente).

Modelo	Parámetros	Exactitud	Entrenamiento	Fuente
2 Conv+pooling+BN	s/d	93,4 %	s/d	Repositorio oficial, tabla de contribuciones
CNN (este trabajo)	**271.210**	**93,06 %**	**418,2 s (26 épocas)**	**Este trabajo**
3 Conv+pooling+2 FC+dropout	s/d	92,6 %	s/d	Repositorio oficial, tabla de contribuciones
2 Conv+pooling	s/d	91,6 %	s/d	Repositorio oficial, tabla de contribuciones
SVC (C=10, kernel=rbf)	s/d	89,7 %	s/d	Xiao, Rasul y Vollgraf (2017), Tabla 3
MLP 256-128-100	s/d	88,33 %	s/d	Repositorio oficial, tabla de contribuciones
MLP de referencia (este trabajo)	**101.770**	**88,23 %**	**22,1 s (33 épocas)**	**Este trabajo**

Predicción sobre imágenes nuevas

El script predecir.py permite usar el modelo con imágenes nuevas: las pasa a escala de grises, las lleva a 28 × 28 píxeles con antialiasing, las invierte si el fondo es claro y devuelve la clase con su top-3. Sobre dos imágenes de prueba exportadas como PNG, una original y otra invertida, reprodujo exactamente las predicciones de la validación, lo que confirma que el preprocesamiento es el mismo.

RESULTADO

La CNN clasifica correctamente 9.306 de las 10.000 imágenes de prueba (93,06 %), con una diferencia de -0,97 puntos porcentuales respecto de la validación y 4,83 puntos por encima de la red sin convolución.

Sus errores se concentran en las prendas de torso (65,27 % del total) y la clase más difícil es Camisa (f1 = 0,792).

6. Conclusiones

El modelo alcanzó una exactitud de 93,06 % sobre las 10.000 imágenes de prueba, 0,97 puntos porcentuales por debajo de la validación, por lo que generaliza a datos que nunca vio. Supera en 4,83 puntos a la red sin convolución entrenada en idénticas condiciones: los filtros aprovechan la estructura espacial de la imagen, que el perceptrón multicapa pierde al aplanarla. Frente al benchmark queda por encima de cuatro de las cinco referencias citadas, incluido el mejor clasificador clásico del artículo original (89,7 %), y por debajo de la variante con normalización por lotes (93,4 %).

Limitaciones. Los errores se concentran en las prendas de torso (65,27 % del total), y Camisa es la clase más débil (f1 = 0,792). El modelo aprendió con imágenes de 28 × 28 píxeles, centradas y con la prenda clara sobre fondo negro, de modo que no generaliza a fotografías reales sin un preprocesamiento equivalente (recorte, centrado, escala de grises e inversión); predecir.py cubre solo una parte de ese proceso. Además, el entrenamiento se hizo solo en CPU: cada corrida demora 6,97 minutos, lo que encarece explorar variantes.

Mejoras posibles. Incorporar normalización por lotes, que en el benchmark lleva una arquitectura de dos convoluciones a 93,4 %; aplicar aumento de datos (data augmentation), con desplazamientos y volteos horizontales, para exponer más variantes de las siluetas de torso que hoy se confunden, y sumar filtros al segundo bloque: hoy la capa densa concentra

el 75,56 % de los parámetros, mientras que las convoluciones, que extraen los detalles que distinguen botones, cuellos o mangas, tienen relativamente poca capacidad.

Anexo A. Entorno y ejecución

Tabla 6. Versiones del entorno y hardware usados en todas las corridas.

Componente	Versión o detalle
Python	3.12.6
TensorFlow	2.21.0
Keras	3.15.1
NumPy	2.5.4
pandas	3.0.6
scikit-learn	1.9.1
Matplotlib	3.11.2
Procesador	Intel(R) Core(TM) i9-10850K CPU @ 3.60GHz
Núcleos	10 físicos, 20 lógicos
Memoria RAM	15,9 GB
GPU	sin GPU (TensorFlow ≥ 2.11 no soporta GPU en Windows nativo)

La Tabla 6 resume las versiones y el hardware de todas las corridas. El video de la cátedra usa Python 3.6 con el paquete keras independiente. Aquí se usa tensorflow.keras (TensorFlow 2.21.0, Keras 3.15.1), que mantiene la misma API: Sequential, Conv2D, MaxPooling2D, Dropout, Flatten, Dense, to_categorical, fit, evaluate, predict y el grabado del modelo. El único cambio visible es la declaración de la entrada con keras.Input. La semilla es 42 en random, NumPy y TensorFlow; sin determinismo forzado de operaciones, en CPU puede haber variaciones mínimas entre corridas.

Comandos para ejecutar cada etapa desde la carpeta tp_cnn:

```
python -m venv .venv
.venv\Scripts\activate            (Linux/Mac: source .venv/bin/activate)
pip install -r requirements.txt
python src/datos.py                 # a. carga, verificación y figuras del dataset
python src/modelo.py                # b. construcción de la red (sin entrenar)
python src/entrenar.py --prueba     # mide una época sin guardar nada
python src/entrenar.py              # c. entrenamiento completo
python src/mlp_referencia.py        # red sin convolución para comparar
python src/validar.py               # d. validación sobre el test oficial
python src/predecir.py imagen.png   # predicción sobre una imagen suelta
python src/generar_informe.py --modulo N --fecha DD/MM/AAAA
```

Estructura del proyecto (codigo_TP_CNN.zip incluye src/, requirements.txt, README.md, el modelo entrenado y el logo; outputs/ se genera al ejecutar los scripts, notebook/ y entrega/ son los entregables y archive/ es la descarga de Kaggle):

```
tp_cnn/src/       datos, modelo, entrenar, mlp_referencia, validar, predecir, generar_informe
tp_cnn/outputs/   resultados.json, historial.json, entorno.txt, modelo .keras, figuras/, pruebas/
tp_cnn/notebook/  TP_CNN_FashionMNIST.ipynb (flujo completo, ejecutado)
tp_cnn/entrega/   informe .docx y .pdf, notebook y código comprimido
archive/          datos descargados de Kaggle (no se incluyen en la entrega)
```

Anexo B. Ejemplos cotejados

Tabla 7. Cotejo de los ejemplos: clase real, clase predicha, probabilidad y segunda opción.

N.º (prueba)	Clase real	Clase predicha	p	2.ª opción (p)	Resultado
881	Remera/top	Remera/top	0,87	Camisa (0,12)	✓
7.868	Remera/top	Remera/top	0,97	Camisa (0,03)	✓
4.476	Pantalón	Pantalón	1,00	Remera/top (0,00)	✓
4.519	Pantalón	Pantalón	1,00	Remera/top (0,00)	✓
670	Buzo	Camisa	0,47	Remera/top (0,33)	✗
6.721	Buzo	Camisa	0,61	Remera/top (0,27)	✗
1.009	Vestido	Vestido	1,00	Camisa (0,00)	✓
5.251	Vestido	Vestido	1,00	Camisa (0,00)	✓
7.304	Abrigo	Abrigo	1,00	Buzo (0,00)	✓
7.510	Abrigo	Abrigo	0,96	Camisa (0,04)	✓
5.215	Sandalia	Sandalia	1,00	Bota (0,00)	✓
7.780	Sandalia	Sandalia	1,00	Zapatilla (0,00)	✓
4.620	Camisa	Camisa	1,00	Abrigo (0,00)	✓
8.519	Camisa	Camisa	0,95	Buzo (0,04)	✓
1.820	Zapatilla	Zapatilla	1,00	Sandalia (0,00)	✓
3.559	Zapatilla	Zapatilla	1,00	Bota (0,00)	✓
6.403	Bolso	Bolso	1,00	Sandalia (0,00)	✓
7.907	Bolso	Bolso	1,00	Remera/top (0,00)	✓
5.630	Bota	Bota	1,00	Sandalia (0,00)	✓
8.205	Bota	Bota	0,89	Sandalia (0,09)	✓

Anexo C. Entrenamiento por época

Tabla 8. Métricas por época de la corrida definitiva (historial.json). Se resaltan la mejor época, cuyos pesos se restauran, y la de detención.

Época	Exactitud entr.	Exactitud val.	Pérdida entr.	Pérdida val.	Segundos
1	0,7503	0,8667	0,6830	0,3676	17,1
2	0,8500	0,8885	0,4152	0,2993	16,5

Época	Exactitud entr.	Exactitud val.	Pérdida entr.	Pérdida val.	Segundos
3	0,8727	0,9078	0,3507	0,2521	18,2
4	0,8868	0,9133	0,3139	0,2355	18,0
5	0,8927	0,9180	0,2955	0,2268	16,7
6	0,8986	0,9248	0,2767	0,2151	16,7
7	0,9030	0,9230	0,2627	0,2145	17,2
8	0,9089	0,9288	0,2495	0,2027	18,8
9	0,9100	0,9265	0,2418	0,2016	16,2
10	0,9145	0,9327	0,2312	0,1951	15,7
11	0,9182	0,9343	0,2233	0,1903	16,2
12	0,9214	0,9330	0,2159	0,1871	17,0
13	0,9230	0,9352	0,2077	0,1828	16,8
14	0,9244	0,9350	0,2009	0,1817	16,0
15	0,9267	0,9343	0,1993	0,1806	16,2
16	0,9300	0,9375	0,1909	0,1840	15,7
17	0,9306	0,9352	0,1865	0,1796	14,4
18	0,9319	0,9362	0,1824	0,1830	15,5
19	0,9330	0,9387	0,1795	0,1820	14,1
20	0,9351	0,9383	0,1734	0,1778	15,0
21 (mejor)	**0,9355**	**0,9403**	**0,1720**	**0,1754**	**16,0**
22	0,9363	0,9322	0,1679	0,1834	15,2
23	0,9384	0,9363	0,1638	0,1872	14,1
24	0,9379	0,9380	0,1643	0,1856	14,4
25	0,9404	0,9365	0,1602	0,1792	15,6
26 (detención)	*0,9399*	*0,9355*	*0,1566*	*0,1832*	*14,0*

Anexo D. Verificación del conjunto de datos

La descarga de Kaggle incluye el conjunto en formato CSV y en formato idx, y ambos se cotejaron imagen por imagen. Los CSV no reproducen la partición oficial: su entrenamiento contiene 7.885 imágenes de la prueba oficial y 4.813 ausentes del conjunto oficial, su prueba contiene 7.918 imágenes del entrenamiento oficial, y ambos comparten 10 imágenes. Los archivos idx usados tienen las sumas MD5 de la Tabla 9, que coinciden con las de los archivos del repositorio oficial descomprimidos, cuyos .gz tienen a su vez los MD5 que publica el repositorio. El diagnóstico completo está en resultados.json (sección dataset).

Tabla 9. Sumas MD5 de los archivos idx usados.

Archivo	MD5
train-images-idx3-ubyte	f4a8712d7a061bf5bd6d2ca38dc4d50a
train-labels-idx1-ubyte	9018921c3c673c538a1fc5bad174d6f9
t10k-images-idx3-ubyte	8181f5470baa50b63fa0f6fddb340f0a
t10k-labels-idx1-ubyte	15d484375f8d13e6eb1aabb0c3f46965